Step 1: Pre-process the Dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

df = pd.read_csv("uber.csv")
print(df.head())
print(df.info())
```

```
   Unnamed: 0                            key  fare_amount  \
0    24238194    2015-05-07 19:52:06.0000003          7.5
1    27835199    2009-07-17 20:04:56.0000002          7.7
2    44984355   2009-08-24 21:45:00.00000061         12.9
3    25894730    2009-06-26 08:22:21.0000001          5.3
4    17610152  2014-08-28 17:47:00.000000188         16.0

           pickup_datetime  pickup_longitude  pickup_latitude  \
0  2015-05-07 19:52:06 UTC        -73.999817        40.738354
1  2009-07-17 20:04:56 UTC        -73.994355        40.728225
2  2009-08-24 21:45:00 UTC        -74.005043        40.740770
3  2009-06-26 08:22:21 UTC        -73.976124        40.790844
4  2014-08-28 17:47:00 UTC        -73.925023        40.744085

   dropoff_longitude  dropoff_latitude  passenger_count
0         -73.999512         40.723217                1
1         -73.994710         40.750325                1
2         -73.962565         40.772647                1
3         -73.965316         40.803349                3
4         -73.973082         40.761247                5
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200000 entries, 0 to 199999
Data columns (total 9 columns):
 #   Column             Non-Null Count   Dtype
---  ------             --------------   -----
 0   Unnamed: 0         200000 non-null  int64
 1   key                200000 non-null  object
 2   fare_amount        200000 non-null  float64
 3   pickup_datetime    200000 non-null  object
 4   pickup_longitude   200000 non-null  float64
 5   pickup_latitude    200000 non-null  float64
 6   dropoff_longitude  199999 non-null  float64
 7   dropoff_latitude   199999 non-null  float64
 8   passenger_count    200000 non-null  int64
dtypes: float64(5), int64(2), object(2)
memory usage: 13.7+ MB
None
```

```
df.drop(['Unnamed: 0', 'key'], axis=1, inplace=True)
```

```
df.isnull().sum()
df.dropna(inplace=True)
```

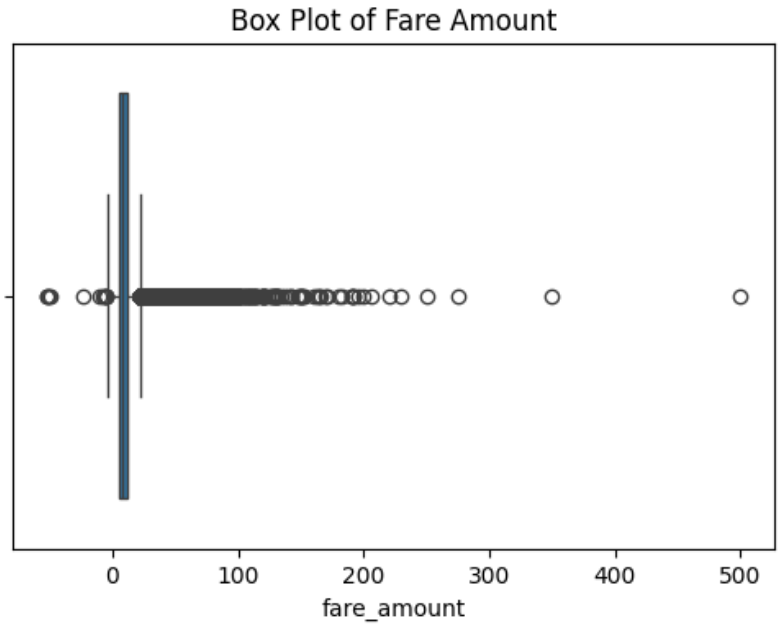
Convert pickup_datetime into useful features

```
df['pickup_datetime'] = pd.to_datetime(df['pickup_datetime'])

df['hour'] = df['pickup_datetime'].dt.hour
df['day'] = df['pickup_datetime'].dt.day
df['month'] = df['pickup_datetime'].dt.month
df['year'] = df['pickup_datetime'].dt.year
```

```
df.drop(['pickup_datetime'], axis=1, inplace=True)
```

Identify Outliers (Box Plot)

```
plt.figure(figsize=(6,4))
sns.boxplot(x=df['fare_amount'])
plt.title("Box Plot of Fare Amount")
plt.show()
```



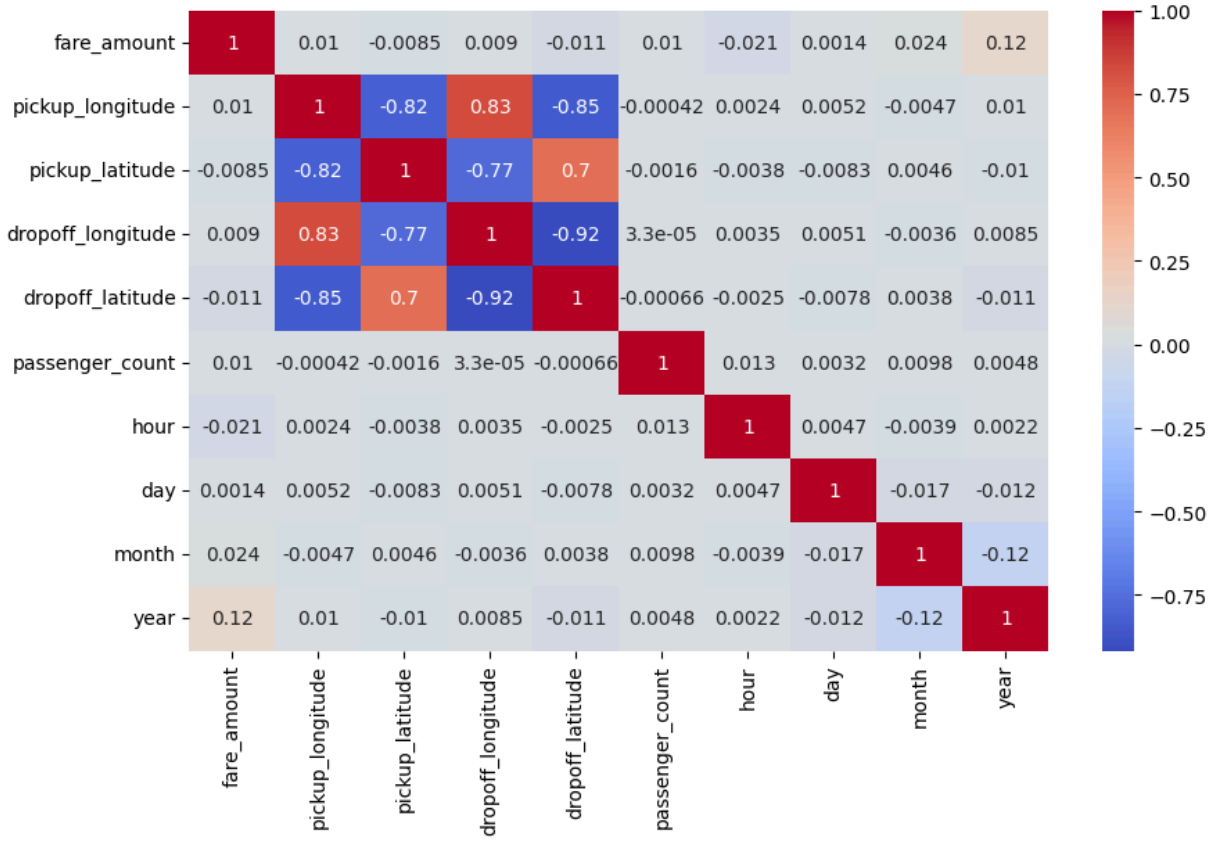
Check the Correlation

```
df.corr()
#df.corr(method='pearson')
#df.corr(method='spearman')
#df.corr(method='kendall')
```

	fare_amount	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latit
fare_amount	1.000000	0.010458	-0.008482	0.008986	-0.011
pickup_longitude	0.010458	1.000000	-0.816461	0.833026	-0.846
pickup_latitude	-0.008482	-0.816461	1.000000	-0.774787	0.702
dropoff_longitude	0.008986	0.833026	-0.774787	1.000000	-0.917
dropoff_latitude	-0.011014	-0.846324	0.702367	-0.917010	1.000
passenger_count	0.010158	-0.000415	-0.001559	0.000033	-0.000
hour	-0.021463	0.002432	-0.003821	0.003478	-0.002
day	0.001384	0.005183	-0.008263	0.005055	-0.007
month	0.023813	-0.004665	0.004625	-0.003605	0.003
year	0.118331	0.009966	-0.010234	0.008467	-0.011

heat map

```
plt.figure(figsize=(10,6))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm')
plt.show()
```



Implement Linear, Ridge, and Lasso Regression

```
X = df.drop('fare_amount', axis=1)
y = df['fare_amount']
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Linear regression

Ridge regression

```
from sklearn.linear_model import Ridge

ridge = Ridge(alpha=1.0)
ridge.fit(X_train, y_train)
y_pred_ridge = ridge.predict(X_test)
```

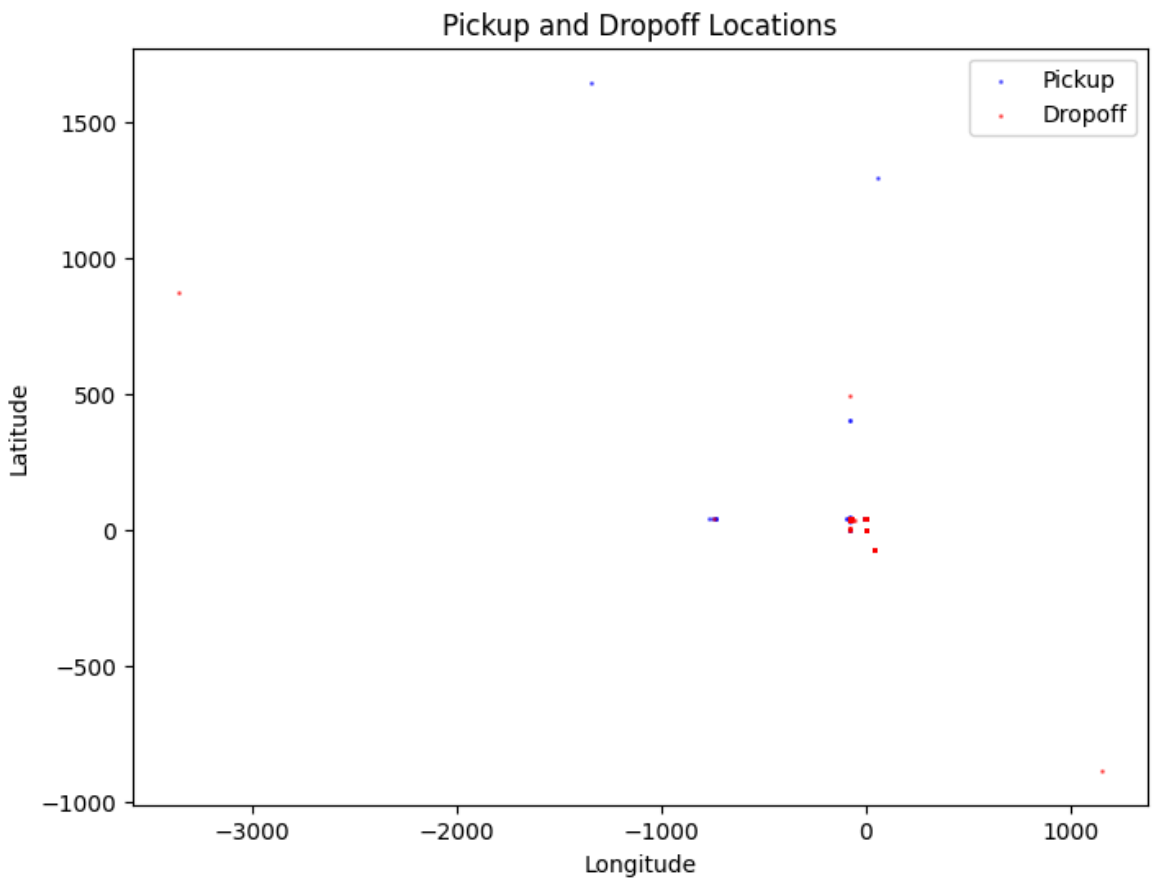
Lasso Regression

```
from sklearn.linear_model import Lasso

lasso = Lasso(alpha=0.1)
lasso.fit(X_train, y_train)
y_pred_lasso = lasso.predict(X_test)
```

Evaluate and Compare Models

scatter Plot

```
import pandas as pd
import matplotlib.pyplot as plt

# Drop rows with missing coordinates
df_clean = df.dropna(subset=['pickup_latitude', 'pickup_longitude', 'dropoff_latitude', 'dropoff_

# Scatter plot of pickup and dropoff locations
plt.figure(figsize=(8,6))
plt.scatter(df_clean['pickup_longitude'], df_clean['pickup_latitude'],
            s=1, alpha=0.5, c='blue', label='Pickup')
plt.scatter(df_clean['dropoff_longitude'], df_clean['dropoff_latitude'],
            s=1, alpha=0.5, c='red', label='Dropoff')

plt.title('Pickup and Dropoff Locations')
plt.xlabel('Longitude')
plt.ylabel('Latitude')
plt.legend()
plt.show()
```



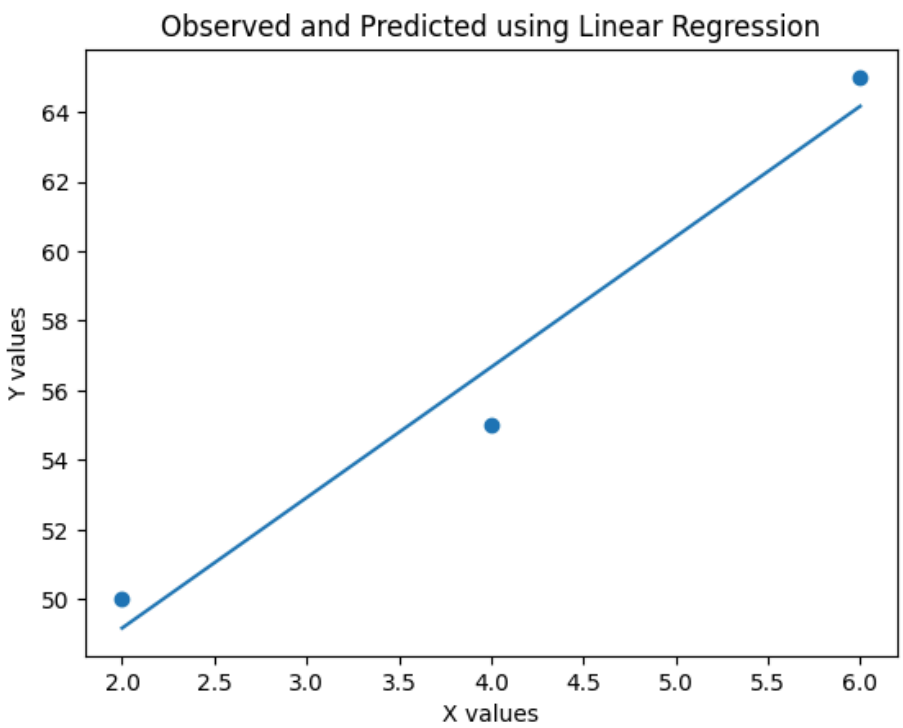
linear Regression

```
import numpy as np
import pandas as pd

# Given data
x = np.array([2, 4, 6])
y = np.array([50, 55, 65])

# Linear regression
m, b = np.polyfit(x, y, 1)
y_pred = m * x + b

# Create table
table = pd.DataFrame({
    "X Values": x,
    "Observed Y": y,
    "Predicted Y": np.round(y_pred, 2)
})

print(table)
```

```
   X Values  Observed Y  Predicted Y
0         2          50        49.17
1         4          55        56.67
2         6          65        64.17
```

scatter plot

```
# Draw scatter plot and regression line
plt.scatter(x, y)
plt.plot(x, y_pred)
plt.xlabel("X values")
plt.ylabel("Y values")
plt.title("Observed and Predicted using Linear Regression")
plt.show()
```



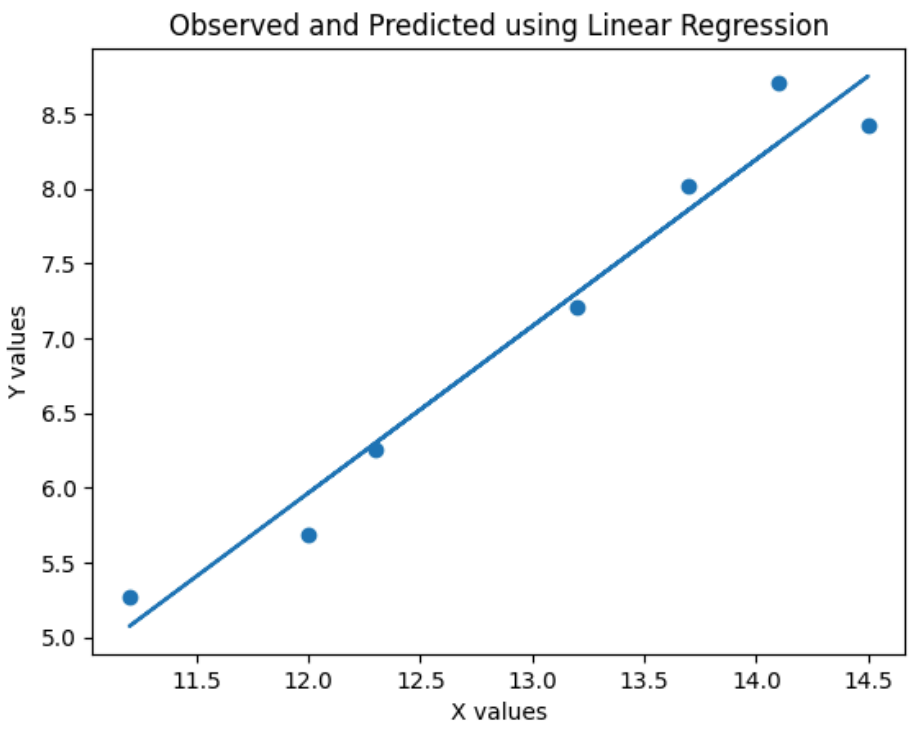
second data set

```
import numpy as np
import pandas as pd

# Given data
x = np.array([12.3,13.7,14.5,11.2,13.2,14.1,12.0])
y = np.array([6.25,8.02,8.42,5.27,7.21,8.71,5.68])

# Linear regression
m, b = np.polyfit(x, y, 1)
y_pred = m * x + b

# Create table
table = pd.DataFrame({
    "X Values": x,
    "Observed Y": y,
    "Predicted Y": np.round(y_pred, 2)
})

print(table)
```

```
   X Values  Observed Y  Predicted Y
0      12.3        6.25         6.30
1      13.7        8.02         7.86
2      14.5        8.42         8.75
3      11.2        5.27         5.07
4      13.2        7.21         7.30
5      14.1        8.71         8.31
6      12.0        5.68         5.96
```

Scatter Plot

```
# Draw scatter plot and regression line
plt.scatter(x, y)
plt.plot(x, y_pred)
plt.xlabel("X values")
plt.ylabel("Y values")
plt.title("Observed and Predicted using Linear Regression")
plt.show()
```




```
import numpy as np
```

```
import pandas as pd
import matplotlib.pyplot as plt

# Given data
x = np.array([2, 4, 6])
y = np.array([50, 55, 65])

# Step 1: Calculate means
x_mean = np.mean(x)
y_mean = np.mean(y)

# Step 2: Calculate slope (b1)
b1 = np.sum((x - x_mean) * (y - y_mean)) / np.sum((x - x_mean)**2)

# Step 3: Calculate intercept (b0)
b0 = y_mean - b1 * x_mean

# Step 4: Predicted values using y = b0 + b1x
y_pred = b0 + b1 * x

# Step 5: Display results
print("Slope (b1):", round(b1, 2))
print("Intercept (b0):", round(b0, 2))

table = pd.DataFrame({
    "X Values": x,
    "Observed Y": y,
    "Predicted Y": np.round(y_pred, 2)
})

print("\nObserved vs Predicted Table:")
print(table)
```

```
Slope (b1): 3.75
Intercept (b0): 41.67

Observed vs Predicted Table:
   X Values  Observed Y  Predicted Y
0         2          50        49.17
1         4          55        56.67
2         6          65        64.17
```

```
import numpy as np
import pandas as pd

# Given data
x = np.array([12.3, 13.7, 14.5, 11.2, 13.2, 14.1, 12.0])
y = np.array([6.25, 8.02, 8.42, 5.27, 7.21, 8.71, 5.68])

# Step 1: Calculate means
x_mean = np.mean(x)
y_mean = np.mean(y)

# Step 2: Calculate slope (b1)
b1 = np.sum((x - x_mean) * (y - y_mean)) / np.sum((x - x_mean)**2)

# Step 3: Calculate intercept (b0)
b0 = y_mean - b1 * x_mean

# Step 4: Predicted values using y = b0 + b1x
y_pred = b0 + b1 * x

# Step 5: Display results
print("Slope (b1):", round(b1, 4))
print("Intercept (b0):", round(b0, 4))
```

```
table = pd.DataFrame({
    "X Values": x,
    "Observed Y": y,
    "Predicted Y": np.round(y_pred, 2)
})

print("\nObserved vs Predicted Table:")
print(table)
```

```
Slope (b1): 1.1154
Intercept (b0): -7.4198

Observed vs Predicted Table:
   X Values  Observed Y  Predicted Y
0      12.3        6.25         6.30
1      13.7        8.02         7.86
2      14.5        8.42         8.75
3      11.2        5.27         5.07
4      13.2        7.21         7.30
5      14.1        8.71         8.31
6      12.0        5.68         5.96
```